

Test Strategy Document

SauceDemo E-commerce Platform

Project: SauceDemo Test Automation

Document Owner: Victor Murashev

1. Project Summary

This document describes the test strategy for SauceDemo e-commerce website. The goal is to create manual test cases first, then convert them into automated ones using modern tools, and check all the main functions of the application.

2. Application Information

Application: SauceDemo (<https://www.saucedemo.com>)

Type: E-commerce website

Main functions: Login, product catalog, shopping cart, checkout

Project time: 8 weeks

Team: 1 QA Engineer

3. Test Approach

3.1 Test Types

Functional Testing:

- User login and logout
- Product browsing
- Shopping cart operations
- Order checkout process
- Different user types

Non-Functional Testing:

- Page loading speed
- Cross-browser testing
- Basic security checks
- Mobile view testing

3.2 Testing Levels

Component Testing:

- Individual page testing
- Form validation

- Button functionality

Integration Testing:

- User workflow testing
- Data flow between pages
- Complete user journey

System Testing:

- End-to-end scenarios
- Full business processes
- All user types testing

4. What Will Be Tested

4.1 In Scope

- Complete shopping process from login to order
- All user types (standard, locked, problem user)
- Different browsers (Chrome, Firefox, Edge)
- Mobile responsive design
- Error handling
- Basic performance testing

4.2 Out of Scope

- Database testing (no access)
- Payment system testing (not real)
- Advanced security testing
- Load testing with many users
- API testing (not available)

5. Test Environment

5.1 Testing Environment

Development Setup:

- Local computer with modern browser
- Internet connection for accessing demo site
- Test automation tools installed

Test Environment:

- SauceDemo website (stable demo environment)
- No special setup needed
- Available 24/7

5.2 Test Data

User Data:

- Standard user: standard_user / secret_sauce
- Locked user: locked_out_user / secret_sauce
- Problem user: problem_user / secret_sauce
- Performance user: performance_glitch_user / secret_sauce

Product Data:

- 6 products available on demo site
- Fixed prices and descriptions
- No need to create test data

6. Tools and Technology

6.1 Main Tools

Test Automation: Playwright with JavaScript/TypeScript

- Reason: Modern tool, good for web testing
- Benefits: Works with all browsers, stable selectors

Alternative: Selenium WebDriver

- Reason: Popular tool, many examples available
- Use case: If Playwright has problems

BDD Tool: Cucumber.js

- Reason: Makes tests readable for business people
- Benefits: Tests look like normal language

6.2 Support Tools

Code Management: Git + GitHub **Documentation:** Markdown files **Reports:** HTML reports, screenshots **CI/CD:** GitHub Actions (basic setup)

7. Risks and Problems

7.1 Main Risks

Risk: Demo website might be unstable

- **Solution:** Check website status before testing, have backup plan

Risk: Browser updates might break tests

- **Solution:** Use stable browser versions, update tests when needed

Risk: Test data might change

- **Solution:** Use provided test users, don't depend on specific product data

Risk: Time limits for learning new tools

- **Solution:** Start with simple tests, add complexity step by step

7.2 Technical Risks

Risk: Automation tools might be difficult to learn

- **Solution:**
 - Read official documentation.
 - Start with simple examples first.
 - Ask AI assistants (like ChatGPT) for clear explanations and examples.

Risk: Tests might be unstable (sometimes pass, sometimes fail)

- **Solution:**
 - Use built-in waiting methods from Playwright (like `waitForSelector`).
 - Do not use fixed waiting times (`sleep`). They often make tests unreliable.

8. Success Criteria

8.1 Entry Criteria

- SauceDemo website is available
- Test tools are installed and working
- Basic test framework is ready
- Test data is available

8.2 Exit Criteria

- At least 50 test cases are automated

- All main functions are tested
- Tests run successfully in different browsers
- Test reports are created
- Documentation is complete

9. Defect Management

9.1 Defect Types

Critical: Login doesn't work, website is down **High:** Shopping cart doesn't work, checkout fails

Medium: Some buttons don't work, wrong text **Low:** Small visual problems, spelling mistakes

9.2 Defect Process

1. Find defect → 2. Write defect report → 3. Add screenshot → 4. Check if it's real bug or test problem

Note: SauceDemo is demo website, so real defects will be documented but not fixed.

10. Test Metrics

10.1 What to Measure

Coverage:

- How many functions are tested: Target 100%
- How many test cases work: Target 95%
- How many browsers tested: Target 3 browsers

Quality:

- How many defects found: Document all
- How fast tests run: Target under 30 minutes
- How stable tests are: Target 90% pass rate

10.2 Reporting

Daily: Quick test run results **Weekly:** Progress report with numbers

End of project: Final summary with lessons learned

11. Project Phases

11.1 Phase 1: Setup (Week 1-2)

- Install and learn tools
- Create basic test structure
- Write first simple tests
- Set up reporting

11.2 Phase 2: Main Testing (Week 3-6)

- Create all test cases
- Test in different browsers
- Fix problems and improve tests
- Add more complex scenarios

11.3 Phase 3: Finish (Week 7-8)

- Complete documentation
- Final test runs
- Create summary report
- Prepare handover materials

12. Success Measurement

The project will be successful when:

- 50+ test cases are working
- All main website functions are tested
- Tests work in 3 different browsers
- Complete documentation is ready
- Knowledge can be shared with others